CE 879: Information Security Mng. and Eng.
Homework 1

Ali Akhgari

Contents

1 Hands-on Practice with TPM 2.0 **3**
- 1.1 Setup and TPM communication . . . 3
  - 1.1.1 Running swtpm . . . 3
  - 1.1.2 Setting the Environment Variable . . . 4
  - 1.1.3 Confirming Communication . . . 4
- 1.2 Change Endorsement Key and Create an Attestation Identity Key . . . 4
  - 1.2.1 Capture the Original EK . . . 4
  - 1.2.2 Wipe State and Generate New TPM Identity . . . 5
  - 1.2.3 Capture the New EK . . . 5
  - 1.2.4 Show the Identity Changed . . . 6
  - 1.2.5 Explanation of Key Commands . . . 6
    - 1.2.5.1 swtpm_setup . . . 6
    - 1.2.5.2 swtpm socket . . . 7
    - 1.2.5.3 tpm2_startup -c . . . 7
    - 1.2.5.4 tpm2_createek . . . 8
    - 1.2.5.5 tpm2_readpublic . . . 8
  - 1.2.6 Create an Attestation Identity Key (AIK) and Make it Persistent . 8
    - 1.2.6.1 Step 1: Set the TCTI Environment Variable . . . 8
    - 1.2.6.2 Step 2: Create a Primary Key (Parent Object) . . . 9
    - 1.2.6.3 Step 3: Create the AIK . . . 9
    - 1.2.6.4 Step 4: Load the AIK into TPM Memory . . . 9
    - 1.2.6.5 Step 5: Make the AIK Persistent . . . 9
    - 1.2.6.6 Step 6: Verify Persistence . . . 10
- 1.3 Sealing and Unsealing on a PCR . . . 10
  - 1.3.1 Initial Setup: Create Primary Key and Define PCR Policy . . . 10
  - 1.3.2 Seal the Secret . . . 10
  - 1.3.3 Load the Sealed Object . . . 10
  - 1.3.4 Situation 1: PCR Holds the Correct Value (STATE_HEALTHY) . 11
  - 1.3.5 Situation 2: PCR Changed to STATE_MALICIOUS . . . 11
  - 1.3.6 Situation 3: PCR Restored to the Original Value . . . 11

2 Theoretical Questions **12**
2.1 TPM 2.0 Hierarchies and Key Management 12
2.2 PCRs and Remote Attestation 12
2.3 Authorization Mechanisms and Policies 13

1 Hands-on Practice with TPM 2.0

1.1 Setup and TPM communication

1.1.1 Running swtpm

```
swtpm socket \
  --tpm2 \
  --tpmstate dir=~/mytpm \
  --ctrl type=tcp,port=2322 \
  --server type=tcp,port=2321 \
  --flags not-need-init &

# [1] 108813
```

This command launches `swtpm` as a background process with the following arguments:

- `socket` — Specifies that the TPM will communicate via TCP sockets.
- `-tpm2` — Instructs `swtpm` to emulate a TPM 2.0 chip. Without this flag, it would default to TPM 1.2.
- `-tpmstate dir=~/mytpm` — Defines the directory where the TPM's persistent state (keys, NVRAM, PCR values, etc.) will be stored.
- `-ctrl type=tcp,port=2322` — Opens a control interface on TCP port 2322. This interface accepts management commands (e.g., sending `TPM_Startup`, resetting the TPM, or saving state).
- `-server type=tcp,port=2321` — Opens the main TPM command interface on TCP port 2321. This is where `tpm2-tools` will connect to send normal TPM commands.
- `-flags not-need-init` — Automatically initializes the TPM on first use, eliminating the need to run `tpm2_startup` separately. Without this flag, the simulator starts uninitialized and rejects all commands except `TPM_Startup`.
- `&` — The ampersand runs the command as a background job in the shell.

The output provides the following information:

- `[1]` — The job number assigned to this background process.
- `108813` — The PID of the `swtpm` process. This can be used to monitor the process or to terminate it.

1.1.2 Setting the Environment Variable

```
export TPM2TOOLS_TCTI="mssim:host=127.0.0.1,port=2321"
```

This environment variable tells `tpm2-tools` to communicate with the `swtpm` simulator via the MSSIM (Microsoft TPM 2.0 Simulator Interface) protocol on localhost port 2321.

1.1.3 Confirming Communication

```
tpm2_getrandom 4 --hex
```

Output:

```
609e116b
```

This command requests 4 random bytes from the TPM's hardware random number generator. The `hex` flag formats the output as a single hexadecimal string.

1.2 Change Endorsement Key and Create an Attestation Identity Key

By rebuilding the TPM state, we change the hardware identity of the TPM so that the public Endorsement Key (EK) before and after the change differ.

1.2.1 Capture the Original EK

First, start a fresh TPM instance and capture the initial EK:

```
rm -rf /tmp/tpmstate && mkdir -p /tmp/tpmstate

swtpm_setup --tpm2 \
  --tpmstate /tmp/tpmstate \
  --create-ek-cert \
  --create-platform-cert \
  --lock-nvram \
  --overwrite

swtpm socket --tpm2 \
  --tpmstate dir=/tmp/tpmstate \
  --ctrl type=tcp,port=2322 \
  --server type=tcp,port=2321 \
  --flags not-need-init &

export TPM2TOOLS_TCTI="swtpm:host=127.0.0.1,port=2321"
tpm2_startup -c

```

```
tpm2_createek -c ek_before.ctx -G rsa -u ek_before.pub
tpm2_readpublic -c ek_before.ctx -f pem -o ek_before.pem
```

Original EK (before rebuild):


```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAm0/rwQ1Phec1gfxZ26F5
cQcCtI9Q1q7IeG1GSiPlVoBC+KVMNa0Tbl6sAoJMvY0717yt/eaq58Ln0v033s3W
obdONoqxw1uGVXO839M5Zw+RxJ6TjrQz+OBt85TsgEjH4KSe8l3HDKoopoxStfHc
7Y+/Sy/e5Sfl2tIolL9RcrdPqfH3WvmEHafy086kIP+2Xer7UOoY8q7RRt3he62L
5+8mP8tW8KY4AA0L/2X28ltN34qwj76NqMcW8qdwmH/XpYwaKYFH2jSmCZVh9epl
9QXE1G34iEvQmA7+yHIh23V6NN9rLlCb1oo7ad1/SPVXiaTQ8oCtrc4NwjFCxq5C
ewIDAQAB
-----END PUBLIC KEY-----
```


1.2.2 Wipe State and Generate New TPM Identity

Kill the existing simulator and recreate the TPM state to generate a new hardware identity:

```
killall -9 swtpm
rm -rf /tmp/tpmstate && mkdir -p /tmp/tpmstate

swtpm_setup --tpm2 \
  --tpmstate /tmp/tpmstate \
  --create-ek-cert \
  --create-platform-cert \
  --lock-nvram \
  --overwrite

swtpm socket --tpm2 \
  --tpmstate dir=/tmp/tpmstate \
  --ctrl type=tcp,port=2322 \
  --server type=tcp,port=2321 \
  --flags not-need-init &

tpm2_startup -c
```

1.2.3 Capture the New EK

```
tpm2_createek -c ek_after.ctx -G rsa -u ek_after.pub
tpm2_readpublic -c ek_after.ctx -f pem -o ek_after.pem
```

New EK (after rebuild):

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0UdIbylnUM9PjTtQBLvq
3HFdeX/gkJBOeoUBAY5APMAiZHpK1+8ryiARMkYCrxtmGTHCsihWs6mD7TIrKpmO
rVhKUxJ9HrrzpnswK9l/3qS8oQx3YeIku+jVdvyBF19GNf/+1yxvGmGriLIx4iQp
VyWI201DzVQxo7IR/HB3eQZm0NcLzYJefh36iyYcsQojYbUJnzvGIpqN1Dea4+IO
24bR7xBnHfoxyj01MtcMzHVDKm2uNnsFcveMT09RF4GIwc9NwVMNlvPqvxMcEAv/
lJPMFtED4Vrd4WYzUBas5AcEIYTbtN7S0cMyOxr1Qs6MHSFpPQUpW5igid5hkpnn
YQIDAQAB
-----END PUBLIC KEY-----
```


1.2.4 Show the Identity Changed

```
diff ek_before.pem ek_after.pem
```

Output (showing difference):


```
2,8c2,8
< MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAm0/rwQ1Phec1gfxZ26F5
< cQcCtI9Q1q7IeG1GSiPlVoBC+KVMNa0Tbl6sAoJMvY0717yt/eaq58Ln0v033s3W
< obdONoqxw1uGVXO839M5Zw+RxJ6TjrQz+OBt85TsgEjH4KSe8l3HDKoopoxStfHc
< 7Y+/Sy/e5Sfl2tIolL9RcrdPqfH3WvmEHafy086kIP+2Xer7UOoY8q7RRt3he62L
< 5+8mP8tW8KY4AA0L/2X28ltN34qwj76NqMcW8qdwmH/XpYwaKYFH2jSmCZVh9epl
< 9QXE1G34iEvQmA7+yHIh23V6NN9rLlCb1oo7ad1/SPVXiaTQ8oCtrc4NwjFCxq5C
< ewIDAQAB
---
> MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0UdIbylnUM9PjTtQBLvq
> 3HFdeX/gkJBOeoUBAY5APMAiZHpK1+8ryiARMkYCrxtmGTHCsihWs6mD7TIrKpmO
> rVhKUxJ9HrrzpnswK9l/3qS8oQx3YeIku+jVdvyBF19GNf/+1yxvGmGriLIx4iQp
> VyWI201DzVQxo7IR/HB3eQZm0NcLzYJefh36iyYcsQojYbUJnzvGIpqN1Dea4+IO
> 24bR7xBnHfoxyj01MtcMzHVDKm2uNnsFcveMT09RF4GIwc9NwVMNlvPqvxMcEAv/
> lJPMFtED4Vrd4WYzUBas5AcEIYTbtN7S0cMyOxr1Qs6MHSFpPQUpW5igid5hkpnn
> YQIDAQAB
```


1.2.5 Explanation of Key Commands

1.2.5.1 swtpm_setup

```
swtpm_setup --tpm2 \
  --tpmstate /tmp/tpmstate \
  --create-ek-cert \
  --create-platform-cert \
  --lock-nvram \
  --overwrite
```

This command initializes and manufactures a TPM 2.0 state directory before the simulator starts:

- `-tpm2` — Targets TPM 2.0.
- `-tpmstate /tmp/tpmstate` — Specifies the directory where the TPM's persistent state will be stored.
- `-create-ek-cert` — Generates an X.509 certificate for the EK. This certificate serves as a platform credential.
- `-create-platform-cert` — Creates a Platform Certificate that binds the TPM to the platform (motherboard/firmware).
- `-lock-nvram` — Locks the NVRAM areas containing these certificates to prevent unauthorized modification after manufacturing.
- `-overwrite` — Overwrites any existing TPM state in the target directory, ensuring a clean, fresh TPM instance.

This command effectively manufactures a TPM with a unique identity (EK and associated certificates).

1.2.5.2 swtpm socket

```
swtpm socket --tpm2 \
  --tpmstate dir=/tmp/tpmstate \
  --ctrl type=tcp,port=2322 \
  --server type=tcp,port=2321 \
  --flags not-need-init &
```

This command launches the `swtpm` simulator as a background process. The arguments are explained in detail in Section 1.1.1.

1.2.5.3 tpm2_startup -c

```
tpm2_startup -c
```

Sends the `TPM2_Startup` command with the `TPM_SU_CLEAR` flag to initialize the TPM. This is required before any TPM commands can be processed. The `-c` flag stands for "clear" (startup with a clear boot). Without this, the TPM remains in an uninitialized state and rejects all other commands.

1.2.5.4 tpm2_createek

```
tpm2_createek -c ek_before.ctx -G rsa -u ek_before.pub
```

Creates the EK, which is a persistent key burned into the TPM that uniquely identifies it:

- `-c ek_before.ctx` — Saves the created key's context to a file (ek_before.ctx) for later use.
- `-G rsa` — Specifies the key type as RSA.
- `-u ek_before.pub` — Extracts and saves the public portion of the EK to a file.

The EK is a restricted decryption key that serves as the root of trust for the TPM's identity. It is never used directly for attestation but can be certified by a manufacturer.

1.2.5.5 tpm2_readpublic

```
tpm2_readpublic -c ek_before.ctx -f pem -o ek_before.pem
```

Reads the public area of a key loaded into the TPM and saves it in a human-readable format:

- `-c ek_before.ctx` — Specifies the key context file (the EK we just created).
- `-f pem` — Formats the output as PEM.
- `-o ek_before.pem` — Writes the PEM-formatted public key to the specified output file.

This allows us to view and compare the EK's public key before and after rebuilding the TPM state.

1.2.6 Create an Attestation Identity Key (AIK) and Make it Persistent

The AIK is a restricted signing key used for TPM attestation operations. It must be created under a primary key and can be made persistent to survive TPM reboots.

1.2.6.1 Step 1: Set the TCTI Environment Variable

```
export TPM2TOOLS_TCTI="mssim:host=127.0.0.1,port=2321"
```

1.2.6.2 Step 2: Create a Primary Key (Parent Object)

The AIK must reside under a parent key in the TPM hierarchy:

```
sudo tpm2_createprimary -C o -c primary.ctx
```

This creates a primary key in the Owner hierarchy (`-C o`). The key context is saved to `primary.ctx`.

1.2.6.3 Step 3: Create the AIK

```
sudo tpm2_create \
  -C primary.ctx \
  -G rsa \
  -u aik.pub \
  -r aik.priv
```

This generates:

- `aik.pub` — The public portion of the AIK
- `aik.priv` — The private portion (stored in TPM format)

1.2.6.4 Step 4: Load the AIK into TPM Memory

```
sudo tpm2_load \
  -C primary.ctx \
  -u aik.pub \
  -r aik.priv \
  -c aik.ctx
```

The AIK is now loaded into TPM memory and accessible via the context file `aik.ctx`.

1.2.6.5 Step 5: Make the AIK Persistent

To make the AIK survive TPM reboots and simulator restarts, evict it to a persistent handle:

```
sudo tpm2_evictcontrol \
  -C o \
  -c aik.ctx \
  0x81010002
```

The handle `0x81010002` is in the persistent handle range (`0x81000000 - 0x81FFFFFF`). The output `persistent-handle: 0x81010002` confirms success.

1.2.6.6 Step 6: Verify Persistence

```
sudo tpm2_readpublic -c 0x81010002
```

Output (abbreviated):

```
name: 000be6a321bb4f06abb5f65bbe98471651f8ca6a834a1f0201a667a03242636b837c
type: rsa
bits: 2048
```

A successful read confirms the AIK is persistent and accessible.

1.3 Sealing and Unsealing on a PCR

Using PCR16, we create a policy based on a value representing `STATE_HEALTHY` and seal the string `FLAG{S4LAB}`. We then demonstrate unsealing in three scenarios: when the PCR holds the correct value, when it is changed to a malicious value, and when it is restored.

1.3.1 Initial Setup: Create Primary Key and Define PCR Policy

First, flush any existing transient objects and create a primary key:

```
sudo tpm2_flushcontext --transient-object
sudo tpm2_createprimary -C o -g sha256 -G rsa -c primary.ctx
```

Next, we assume PCR16 has been extended with a value representing `STATE_HEALTHY`. The policy is implicitly defined by the PCR selection during the sealing operation.

1.3.2 Seal the Secret

The following command seals the string `FLAG{S4LAB}` binding it to the current value of PCR16:

```
echo -n "FLAG{S4LAB}" | sudo tpm2_create \
  -C primary.ctx \
  -L pcr_policy.ctx \
  -i- \
  -u sealed.pub \
  -r sealed.priv
```

1.3.3 Load the Sealed Object

```
sudo tpm2_load -C primary.ctx -u sealed.pub -r sealed.priv -c sealed.ctx
```

1.3.4 Situation 1: PCR Holds the Correct Value (STATE_HEALTHY)

When PCR16 contains the expected value, unsealing succeeds and outputs the flag:

```
sudo tpm2_startauthsession --policy-session -S session.ctx
sudo tpm2_policypcr -S session.ctx -l sha256:16
sudo tpm2_unseal -c sealed.ctx -p session:session.ctx
sudo tpm2_flushcontext session.ctx
```

Output:

```
FLAG{S4LAB}
```

1.3.5 Situation 2: PCR Changed to STATE_MALICIOUS

We extend PCR16 with a different value representing `STATE_MALICIOUS`:

```
echo -n "STATE_MALICIOUS" | sudo tpm2_pcrextend 16:sha256=$(printf "
    STATE_MALICIOUS" | sha256sum | cut -d' ' -f1)
```

Now attempt to unseal:

```
sudo tpm2_startauthsession --policy-session -S session.ctx
sudo tpm2_policypcr -S session.ctx -l sha256:16
sudo tpm2_unseal -c sealed.ctx -p session:session.ctx
```

Output:

```
ERROR: Tss2_Sys_Unseal(0x99D) - tpm:session(1):a policy check failed
```

Why this failure occurs: The sealed secret is cryptographically bound to the specific PCR16 value that existed at sealing time. When the PCR value changes, the computed policy digest no longer matches the authorization policy stored with the sealed object. The TPM rejects the unseal attempt because the current system state (PCR16 = `STATE_MALICIOUS`) does not satisfy the policy that was originally created (`STATE_HEALTHY`).

1.3.6 Situation 3: PCR Restored to the Original Value

We restore PCR16 to the original `STATE_HEALTHY` value:

```
echo -n "STATE_HEALTHY" | sudo tpm2_pcrextend 16:sha256=$(printf "
    STATE_HEALTHY" | sha256sum | cut -d' ' -f1)
```

Now unseal again:

```
sudo tpm2_startauthsession --policy-session -S session.ctx
sudo tpm2_policypcr -S session.ctx -l sha256:16
sudo tpm2_unseal -c sealed.ctx -p session:session.ctx
sudo tpm2_flushcontext session.ctx
```

Output:

```
FLAG{S4LAB}
```

The unseal succeeds again, confirming that the secret can be recovered once the PCR returns to its trusted value.

2 Theoretical Questions

2.1 TPM 2.0 Hierarchies and Key Management

Question 1: What are the four hierarchies defined in TPM 2.0? Briefly explain the purpose of each hierarchy.

- **Platform hierarchy**: Controlled by the OEM (Original Equipment Manufacturer) and platform firmware for boot-time security tasks.
- **Storage hierarchy**: Intended for the end user or IT department to manage general application keys.
- **Endorsement hierarchy**: Designed specifically for privacy-sensitive attestation and device identification.
- **NULL hierarchy**: Acts as an ephemeral cryptographic coprocessor, where keys are wiped upon every reboot.

Question 2: How does having multiple, independent hierarchies improve security compared to TPM 1.2?

> Having multiple independent hierarchies improves security compared to TPM 1.2 (which used a single owner and hierarchy) by allowing separate administrative roles. This means an OEM can rely on the Platform hierarchy being enabled for secure boot functions even if a user clears or disables the Storage hierarchy, ensuring platform-level protections remain intact.

2.2 PCRs and Remote Attestation

Question 1: Describe the role of PCRs in ensuring boot integrity and enabling remote attestation.

> **Platform Configuration Registers (PCRs)** record a cryptographic measurement of the system's software state, spanning from early boot firmware to the operating system. This enables **remote attestation**, a mechanism by which the TPM issues a signed *quote* of PCR values to prove to a remote party that the system is in a trusted, uncompromised state.

Question 2: Explain the extend operation and why it makes PCRs tamper-resistant.

> PCRs are updated using the **extend operation**, which concatenates the current register value with a new measurement and hashes the result. This process is inherently **tamper-resistant**: the one-way nature of the hash makes it computationally infeasible to reverse an extend or restore a previous register state.

Question 3: How does sealing/unsealing rely on PCR values?

> **Sealing and unsealing** leverage these properties by binding the encryption of a secret to a specific PCR state. The TPM will unseal and release the secret only if the current PCR measurements match the *golden values* defined when the data was originally sealed.

2.3 Authorization Mechanisms and Policies

Question 1: Name the three authorization methods supported by TPM 2.0.

> **Password**, **HMAC**, and **Policy (Extended Authorization)**.

Question 2: What is the main advantage of using an Extended Authorization Policy over a simple password?

> The main advantage of using an **Extended Authorization Policy** over a simple password is the ability to enforce **multifactor and conditional security**. While a password only requires knowing a secret, an Extended Authorization Policy can mandate that multiple conditions be met—such as a specific fingerprint match, the machine being in a trusted state (PCR values), or the command originating from a specific locality.

Question 3: Relate this to the sealing/unsealing experiment you performed in Part 3 of the practical section.

> This is demonstrated in my PCR sealing experiment. The secret was sealed to PCR16 = `STATE_HEALTHY`. When PCR16 was changed to `STATE_MALICIOUS`, unsealing failed with a "policy check failed" error. When PCR16 was restored, unsealing succeeded. Unlike a simple password, which only verifies *knowledge*, this Extended Authorization Policy verifies *system state*. An attacker with the correct password cannot unseal the secret if the PCR values do not match the policy.